

Reverse Engineering

RS-485 Checksum Reverse Engineering (Watlow EZ-Zone PM)

Asked 10 years, 5 months ago Modified 6 years, 10 months ago Viewed 3k times

I am trying to reverse engineer a two wire RS-485 standard serial bus interface to talk to a [Watlow EZ-Zone PM](#) of which I have not been able to find any documentation of the protocol. I have managed to figure out most of the hex commands except for the "check bytes" by sniffing the serial communications from the [Labview driver](#) (which doesn't work for my particular application).

I am having trouble figuring out the 3 check bytes. Any help is appreciated.

Example hex command:

Zone										Parameter				Instance	

55	FF	05	10	00	00	06	E8	01	03	01	04	01	01	E3	99
							^^							^^	^^
							check byte				check bytes				

The first check byte only changes with the bytes before it:

```
55 FF 05 10 00 00 06 E8 01 03 01 04 01 01 E3 99
55 FF 05 11 00 00 06 61 01 03 01 04 01 01 E3 99
55 FF 05 12 00 00 06 F9 01 03 01 04 01 01 E3 99
55 FF 05 13 00 00 06 70 01 03 01 04 01 01 E3 99
55 FF 05 14 00 00 06 CA 01 03 01 04 01 01 E3 99
```

The second two bytes only change with the bytes after the first check byte:

```
55 FF 05 10 00 00 06 E8 01 03 01 04 01 01 E3 99
55 FF 05 10 00 00 06 E8 01 03 01 04 02 01 8B B3
55 FF 05 10 00 00 06 E8 01 03 01 04 03 01 53 AA
55 FF 05 10 00 00 06 E8 01 03 01 04 04 01 5B E7
55 FF 05 10 00 00 06 E8 01 03 01 04 05 01 83 FE
55 FF 05 10 00 00 06 E8 01 03 01 05 05 01 5F A4
55 FF 05 10 00 00 06 E8 01 03 01 06 05 01 3B 4B
55 FF 05 10 00 00 06 E8 01 03 01 07 05 01 E7 11
55 FF 05 10 00 00 06 E8 01 03 01 08 05 01 20 5B
55 FF 05 10 00 00 06 E8 01 03 01 09 05 01 FC 01
55 FF 05 10 00 00 06 E8 01 03 01 0A 05 01 98 EE
```

I did find reference to a CRC checksum in the Watlow Modbus documentation. However I have no idea what the polynomial is. Any ideas?

hardware

serial-communication

Share Improve this question Follow

edited Feb 20, 2015 at 21:59

asked Feb 20, 2015 at 21:45



Onlyjus

203 2 7

What would be the command to send the SETPOINT and read the PV? – user12285 Apr 16, 2015 at 22:47

@JorgeCapura I actually programmed this in Labview, discussion and VIs posted over on the Labview Forum: forums.ni.com/t5/Instrument-Control-GPIB-Serial/... – Onlyjus Apr 20, 2015 at 11:22

Is there any source code that I can download? I am trying to talk to it using Linux. To bad they don't make a Linux interface for it. Thanks Monito – Patrick Lamastus Sep 18, 2018 at 13:07

I only programmed it in labview. Watlows dll is windows only. You could probably use python and pyserial. – Onlyjus Sep 28, 2018 at 12:27

1 Answer

Sorted by: Highest score (default)



16



I downloaded the [EZ-ZONE Configurator](#) and reverse engineered it to see how it works.

The serial data you're seeing is actually the [BACnet](#) MS/TP (master-slave/token-passing) protocol. You can find the [Wireshark](#) protocol decoder for it [here](#). However, to save you the time, I'll help you get to the meat of calculating those check bytes.

In BACnet parlance, 55 FF is called the "preamble", the first check byte is called the "Header CRC", the last two check bytes are called the "Data CRC", etc. For simplification though, let's call `b[]` your byte array: `b[0] = 55`, `b[1] = FF`, etc.

The first check byte (a.k.a. "Header CRC") (`b[7]`) is calculated using the BACnet 8-bit CRC as follows.

We first define our CRC table:

```
BYTE crc[256] =
{
    0x00, 0xfe, 0xff, 0x01, 0xfd, 0x03, 0x02, 0xfc,
    0xf9, 0x07, 0x06, 0xf8, 0x04, 0xfa, 0xfb, 0x05,
    0xf1, 0x0f, 0x0e, 0xf0, 0x0c, 0xf2, 0xf3, 0x0d,
    0x08, 0xf6, 0xf7, 0x09, 0xf5, 0x0b, 0x0a, 0xf4,
    0xe1, 0x1f, 0x1e, 0xe0, 0x1c, 0xe2, 0xe3, 0x1d,
    0x18, 0xe6, 0xe7, 0x19, 0xe5, 0x1b, 0x1a, 0xe4,
    0x10, 0xee, 0xef, 0x11, 0xed, 0x13, 0x12, 0xec,
```

```

0xe9, 0x17, 0x16, 0xe8, 0x14, 0xea, 0xeb, 0x15,
0xc1, 0x3f, 0x3e, 0xc0, 0x3c, 0xc2, 0xc3, 0x3d,
0x38, 0xc6, 0xc7, 0x39, 0xc5, 0x3b, 0x3a, 0xc4,
0x30, 0xce, 0xcf, 0x31, 0xcd, 0x33, 0x32, 0xcc,
0xc9, 0x37, 0x36, 0xc8, 0x34, 0xca, 0xcb, 0x35,
0x20, 0xde, 0xdf, 0x21, 0xdd, 0x23, 0x22, 0xdc,
0xd9, 0x27, 0x26, 0xd8, 0x24, 0xda, 0xdb, 0x25,
0xd1, 0x2f, 0x2e, 0xd0, 0x2c, 0xd2, 0xd3, 0x2d,
0x28, 0xd6, 0xd7, 0x29, 0xd5, 0x2b, 0x2a, 0xd4,
0x81, 0x7f, 0x7e, 0x80, 0x7c, 0x82, 0x83, 0x7d,
0x78, 0x86, 0x87, 0x79, 0x85, 0x7b, 0x7a, 0x84,
0x70, 0x8e, 0x8f, 0x71, 0x8d, 0x73, 0x72, 0x8c,
0x89, 0x77, 0x76, 0x88, 0x74, 0x8a, 0x8b, 0x75,
0x60, 0x9e, 0x9f, 0x61, 0x9d, 0x63, 0x62, 0x9c,
0x99, 0x67, 0x66, 0x98, 0x64, 0x9a, 0x9b, 0x65,
0x91, 0x6f, 0x6e, 0x90, 0x6c, 0x92, 0x93, 0x6d,
0x68, 0x96, 0x97, 0x69, 0x95, 0x6b, 0x6a, 0x94,
0x40, 0xbe, 0xbf, 0x41, 0xbd, 0x43, 0x42, 0xbc,
0xb9, 0x47, 0x46, 0xb8, 0x44, 0xba, 0xbb, 0x45,
0xb1, 0x4f, 0x4e, 0xb0, 0x4c, 0xb2, 0xb3, 0x4d,
0x48, 0xb6, 0xb7, 0x49, 0xb5, 0x4b, 0x4a, 0xb4,
0xa1, 0x5f, 0x5e, 0xa0, 0x5c, 0xa2, 0xa3, 0x5d,
0x58, 0xa6, 0xa7, 0x59, 0xa5, 0x5b, 0x5a, 0xa4,
0x50, 0xae, 0xaf, 0x51, 0xad, 0x53, 0x52, 0xac,
0xa9, 0x57, 0x56, 0xa8, 0x54, 0xaa, 0xab, 0x55
};

```

And next we can calculate `b[7]` :

```
b[7] = ~crc[b[6] ^ crc[b[5] ^ crc[b[4] ^ crc[b[3] ^ crc[~b[2]]]]]
```

To calculate the value of the last two check bytes ("Data CRC"):

Perform a CRC-16 of the 6 bytes between the first check byte and the last two check bytes (in your first example, this would be the bytes `01 03 01 04 01 01`), with `0xFFFF` (`-1`) as the initial value for the CRC-16, and `0x8408` as the polynomial. Then bit-flip (a.k.a. "not", a.k.a. "invert") the result and read it in little-endian.

Share Improve this answer Follow

answered Feb 20, 2015 at 23:34



Jason Geffner

20.7k 1 38 75

For anyone curious as to how I determined that this was BACnet MS/TP, I found the non-standard 8-bit CRC table in the data-link driver and Googled for its data. In hindsight, just [Googling for the sample data in the question](#) would have been much faster :) – Jason Geffner Feb 20, 2015 at 23:41

Thanks so much! I spent 2 weeks on and off trying to figure this out and u did it in an hour. lol. Just have to search for the right thing... – Onlyjus Feb 22, 2015 at 14:13

"Just have to search for the right thing..." -- Nope, it's more about reverse engineering the code. The Google search only came up because I wondered if this was a standard protocol. But I never could have extracted the checksum algorithms without reverse engineering the code. – Jason Geffner Feb 22, 2015 at 16:44

The header check byte should be this: $b[7] = 256 + \sim \text{crc}[b[6] \wedge \text{crc}[b[5] \wedge \text{crc}[b[4] \wedge \text{crc}[b[3] \wedge \text{crc}[\sim b[2]]]]]]$ right? missing the 256+ ? Thanks again! – [Onlyjus](#) Feb 23, 2015 at 14:14

The header check byte is a single 8-bit byte, so $256 == 0$. No need to add it to the formula.
– [Jason Geffner](#) Feb 23, 2015 at 14:40 

Start asking to get answers

Find the answer to your question by asking.

[Ask question](#)

Explore related questions

[hardware](#)

[serial-communication](#)

See similar questions with these tags.